

Glasses + Mobile + Watch

Concept Designs & Architecture

User Interaction Modalities

This section is a reference for the ways a user can interact with the Glasses. Each idea in the rest of the document draws on one or more of these modalities. The display is treated as output — once the system shows something, the user's response back is itself one of the modalities below.

Quick Reference

At-a-glance comparison of the six interaction modalities.

Modality	Trigger Style	Always-on / Event / Continuous	Power Profile	Typical Use
Touchpad	Physical trigger (tap / swipe / long-press)	Event-driven	Essentially zero	Quick commands; confirm / dismiss; explicit invocations.
IMU-based gestures	Motion gestures (head nod, QL gestures, watch / ring motion)	Event-driven; lightweight always-on detection	Very low	Subtle, hands-free signals; discreet confirmations; quick yes / no responses.
Voice only (AIT + Hotword)	Hotword detection + wearer-talking classification	Always-on	Low (small classifiers run continuously)	Natural-language requests; queries; dictation; the default entry point for most assistant interactions.
Low-frequency visual understanding	Camera-detected gestures or scene cues at ~1 FPS	Continuous at low FPS; can also be event-driven	Moderate (camera + lightweight vision)	Hand gestures the system sees rather than feels (e.g., framing a region with both hands); spatial-intent signals; scene-aware triggers.
Threshold-based ambient audio	Wake when audio crosses a level / class threshold (announcement, talk event, loud noise)	Always-on at very low sample rate; event-driven escalation	Very low at idle, rises only when a trigger fires	PA announcements; the start of a conversation; safety-relevant sounds; passive context that the system then chooses to act on.
Live AI (high-frequency multimodal)	Continuous 720p ~30 FPS video + audio streamed to a multimodal model	Continuous (session-bounded)	High (camera at 30 FPS + streaming + cloud inference)	True “assistant looking with you” experiences: continuous object tracking, real-time guidance, conversational reasoning over what is in front of you.

Modalities in Depth

1. Touchpad

A physical surface on the Glasses (or paired device) that the user taps, swipes, or long-presses. Strictly event-driven — there is no signal until the user touches it. The simplest and most reliable modality: zero false positives, almost no power cost, and unambiguous intent. Best for short, explicit actions: invoking the assistant on demand, confirming or dismissing a prompt, scrubbing through a list, ending a session. It is also the most boring — it requires a hand free and a deliberate motion, so it does not scale to the kinds of passive, ambient interactions Glasses are uniquely suited for.

2. IMU-based gestures

Motion-sensor input from the Glasses themselves (head nods, taps detected via accelerometer) or from companion devices the user already wears: a paired watch, a smart ring, or QL-style gesture detectors. Detection runs at low cost as a lightweight always-on classifier looking for a small set of pre-defined gesture signatures. Power cost is very low because IMU sampling is cheap and the classifier model is tiny. Best for short, discreet confirmations — a head nod for “yes”, a wrist flick to dismiss — in social contexts where speaking aloud or tapping a device would be awkward. The fixed gesture vocabulary is the limit: it is excellent for binary or small-set choices, but cannot carry rich intent.

3. Voice only (AIT + Hotword)

Always-on audio pipeline running two lightweight models: a hotword detector waiting for the trigger phrase, and an Am-I-Talking (AIT) classifier that distinguishes the wearer’s voice from background speech. Once the hotword fires and AIT confirms the wearer is speaking, the audio is streamed to ASR and the assistant. Power cost is low because both classifiers are small and run on dedicated audio hardware where available. This is the default entry point for most assistant interactions: natural language is the highest-bandwidth way for a user to express intent. Limitations are social (voice is awkward in many settings), acoustic (noisy environments degrade accuracy), and privacy-related (continuous mic capture, even on-device, is a sensitive default).

4. Low-frequency visual understanding

The Glasses camera runs at roughly 1 FPS, feeding a lightweight on-device vision pipeline that watches for gestures the user is making or scene cues that imply intent. Unlike IMU gestures, these are gestures the system sees rather than feels — framing a region with both hands, pointing, holding an object up. Can run continuously at low FPS or be woken event-driven by another signal (a hotword, an audio trigger, a touchpad tap). Power cost is moderate: camera + on-device vision is more expensive than audio or IMU, but 1 FPS is far cheaper than live-AI rates. Best for spatially-grounded intent: the user can direct the system’s attention to a region of their visual field without describing it in words. Limitations include lighting, framing requirements (the gesture must be visible to the camera), and the privacy weight of any always-on camera.

5. Threshold-based ambient audio understanding

A very-low-cost audio pipeline that listens passively and fires only when a configured threshold or class is met: speech begins, a loud event occurs, a PA announcement starts, a specific environmental sound is detected. Distinct from the always-on voice modality — there is no hotword and no expectation of wearer intent. The system is sampling the world and deciding whether something worth acting on is happening. Power cost is very low at idle and rises only when a trigger fires (at which point the camera or a heavier model may be woken). Best for opportunistic context: notifying the wearer about an announcement they might miss, starting a deeper capture pipeline only when there is actually something to capture. Needs careful threshold tuning to avoid false wakes from TV, music, or background chatter.

6. Live AI (high-frequency multimodal)

A session-bounded mode in which the Glasses stream 720p video at roughly 30 FPS together with audio to a multimodal model in the cloud, enabling continuous reasoning over what the wearer is seeing and saying. This is the “true assistant” experience: the model can track objects across frames, follow a procedure visually, answer questions about what just happened. Power cost is high — camera at full rate, streaming, and cloud inference all running simultaneously — so this is not a default state. Sessions are explicitly started (often by voice or touchpad) and explicitly ended. Best for tasks where lower-bandwidth modalities would be too slow or coarse: live tutoring, hands-on guidance, complex multi-step interactions. Limitations are battery, bandwidth, latency, and the privacy implications of continuous upload.

Output Side

Display, audio, and haptics are how the system speaks back to the user. They are not interaction modalities in their own right — the user does not “interact” by being shown something — but they shape which modality the user reaches for next. A glanceable display answer may need no response at all; an audio question naturally invites a voice reply; a haptic alert often pairs with a touchpad or IMU confirmation. Proactive, system-initiated responses (a nudge based on time or context, an alert when something is detected) live on the output side; the user’s reaction to them flows back through one of the six modalities above.

Notes on Combining Modalities

- **Stacking is common:** Most non-trivial interactions blend modalities. A typical request might start with a hotword (voice), be refined by a hand-framing gesture (low-freq visual), and be confirmed by a head nod (IMU).
- **Cost ladder:** Lower-cost modalities should gate higher-cost ones. Audio threshold can wake low-freq vision; low-freq vision can confirm whether to escalate to live AI. Running expensive modalities continuously is rarely necessary.
- **Social legibility:** Some modalities are conspicuous (voice in public, audio cues at low volume), some are discreet (IMU, touchpad). Choosing the right modality is partly a UX decision about what the wearer is comfortable doing in front of others.
- **Passive sensing is separate:** Phone-side sensors (IMU, GPS, Motion Activity API) and similar context signals are not user interaction modalities — the user is not interacting — but they often augment all six by providing the situational context that decides which modality is appropriate, or by triggering capture without any user action at all.

Idea 1 — Gesture-Driven Image Crop for Visual Q&A

Goal

Allow the user to direct the AI assistant’s attention to a specific region of their visual field using an intuitive two-handed close gesture. The cropped region — combined with optional voice and stored user context — becomes the focused input to a multimodal model, producing more relevant and precise answers than a full-frame query.

Why It Is Useful

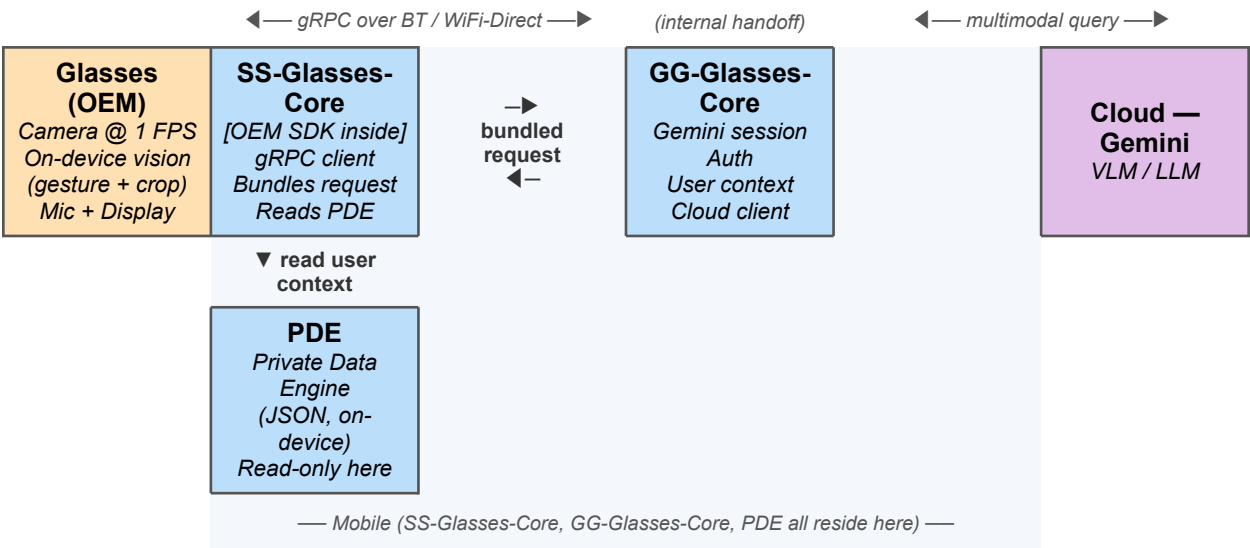
Hands-free, always-on cameras tend to over-capture. The model receives the entire scene and has to guess what the user cares about, which wastes tokens and frequently produces vague or wrong answers. A gesture-defined crop lets the user signal intent spatially — without needing to verbally describe what they are looking at — and dramatically narrows the model’s attention.

Use Cases

- **Shopping / product inspection:** User frames a single item on a crowded shelf with both hands; the assistant identifies it, compares prices, or checks for allergens (intent inferred from PDE-stored user preferences).
- **Reading / translation in the wild:** User crops a paragraph on a foreign-language menu or a sign; the assistant reads or translates only that region — no scene clutter, no ambiguity about which text was meant.

Architecture Diagram (Editable Word Shapes)

All boxes are native Word table cells — fully editable: change text, recolor, resize, or replace with shapes. The light-blue tinted band marks the Mobile group. SS-Glasses-Core hosts the OEM SDK as a sub-module and owns gRPC transport (BT / WiFi-Direct) to the Glasses; GG-Glasses-Core (also on mobile) owns the cloud session.



Legend

	User / actor
--	--------------

	Glasses (on-device)
	Mobile modules
	Watch (n/a for this idea)
	Cloud services

Architecture Diagram (Mermaid Source)

Paste into mermaid.live, [Notion](#), [GitHub](#), or any Mermaid-compatible renderer.

```

flowchart LR
    User([User]) -- "two-handed close gesture<br/>+ optional voice" --> Glasses
    subgraph GlassesNode ["Glasses (OEM)"]
        direction TB
        Cam["Camera @ 1 FPS<br/>frame buffer"]
        Vis["On-device vision<br/>(gesture + crop detect)"]
        Mic["Mic + Display"]
    end
    Glasses --> Vis
    subgraph Mobile ["Mobile"]
        direction TB
        subgraph SSGC ["SSG-Core"]
            OEM["OEM SDK<br/>(sub-module)"]
        end
        GGGC["GGG-Core<br/>(Gemini session,<br/>auth, user context)"]
        PDE["PDE<br/>(Private Data Engine,<br/>JSON, read-only here)"]
    end
    Vis -- "gRPC over<br/>BT / WiFi-Direct" --> OEM
    SSGC -- "read user context" --> PDE
    SSGC -- "bundled request<br/>(image + voice + context)" --> GGGC
    Cloud["Cloud: Gemini VLM/LLM"]
    GGGC -- "multimodal query" --> Cloud
    Cloud -- "answer" --> GGGC
    GGGC -- "answer" --> SSGC
    SSGC -- "gRPC response" --> Vis

```

Architecture Diagram (Eraser.io DSL)

Paste into app.eraser.io as a Diagram-as-Code source. Eraser produces a richer, icon-aware visual than Mermaid out of the box.

```
// Idea 1 — Gesture-Driven Image Crop for Visual Q&A
// Paste into https://app.eraser.io (Diagram-as-Code)

User [icon: user, color: yellow]

Glasses (OEM) [icon: glasses, color: orange] {
  Camera_1FPS [icon: camera]
  Frame_Buffer [icon: layers]
  OnDevice_Vision [icon: eye, label: "Gesture + Crop"]
  Mic_Display [icon: mic]
}

Mobile [icon: smartphone, color: blue] {
  SS_Glasses_Core [icon: cog, label: "SS-Glasses-Core"] {
    OEM_SDK [icon: plug, label: "OEM SDK (sub-module)"]
  }
  GG_Glasses_Core [icon: cog, label: "GG-Glasses-Core\nGemini session, auth, user
ctx"]
  PDE [icon: database, label: "PDE\nUser prefs (JSON)\nRead-only here"]
}

Cloud [icon: cloud, color: purple] {
  Gemini [icon: sparkles, label: "Gemini VLM/LLM"]
}

// Connections
User > Glasses (OEM): two-handed close gesture + optional voice
OnDevice_Vision > OEM_SDK: gRPC over BT / WiFi-Direct\n(cropped image + voice opt)
SS_Glasses_Core > PDE: read user context
SS_Glasses_Core > GG_Glasses_Core: bundled request\n(image + voice + context)
GG_Glasses_Core > Gemini: multimodal query
Gemini > GG_Glasses_Core: answer
GG_Glasses_Core > SS_Glasses_Core: answer
OEM_SDK > OnDevice_Vision: gRPC response
```

End-to-End Flow

- Glasses camera streams at ~1 FPS into a small rolling frame buffer.
- On-device vision watches for the two-handed close gesture; once stable, it locks the crop region and selects the best frame from the buffer.
- Optionally, the Glasses mic captures a short voice utterance bound to the same gesture event.
- Cropped image (+ voice if present) is sent over gRPC (BT or WiFi-Direct) to the OEM SDK inside SS-Glasses-Core on the mobile device.
- SS-Glasses-Core reads the relevant slice of user context from PDE and assembles a bundled request.
- The bundle is handed off to GG-Glasses-Core (also on mobile), which maintains the Gemini session, handles auth, and submits the multimodal query to the cloud.
- The response flows back: Cloud → GG-Glasses-Core → SS-Glasses-Core → (gRPC) → Glasses display + audio. Target end-to-end latency: ~1–2.5 s.

Technical Details to Consider

On-Glasses Vision

- **Frame buffering:** Maintain a rolling buffer of the last N frames at 1 FPS. On gesture detection, pick the sharpest frame with the gesture fully formed — not just the trigger frame.
- **Gesture stability check:** Require the two-handed close gesture to hold for ~300–500 ms before committing, to suppress false positives from incidental hand motion.
- **Crop region refinement:** After gesture detection, snap the bounding box to detected object/text edges within the framed region before sending. Reduces wasted pixels.
- **Power & thermal:** Always-on camera at 1 FPS plus on-device gesture model has a measurable battery/thermal cost — budget for it; consider duty cycling when the user is stationary.

Glasses ↔ Mobile Transport

- **gRPC over BT / WiFi-Direct:** Bidirectional stream between Glasses and SS-Glasses-Core. BT for low-bandwidth control + small payloads; auto-switch to WiFi-Direct for image bursts to keep latency low.
- **Compression:** Send compressed JPEG of the cropped region only — never the full frame. Bandwidth and latency win.
- **Backpressure:** If the gRPC channel saturates, drop to lower image quality before dropping requests. Voice is small and should always go through.

Intent Resolution

Voice is optional, so intent must be derivable from multiple sources. Resolution priority:

- Voice utterance (if present and confidently transcribed)
- Image-only inference at the cloud VLM (e.g., “this is a wine bottle, user probably wants info”)
- PDE user preferences as fallback (e.g., user is allergen-sensitive → default to ingredient check; user is a language learner → default to translate)

Mobile Bundling (SS-Glasses-Core)

- **Voice-image-context binding:** Voice, cropped image, and the relevant PDE slice are bundled with a single session ID and handed to GG-Glasses-Core as one multimodal request.
- **PDE query scope:** SS-Glasses-Core requests only relevant context keys from PDE (e.g., dietary restrictions, languages, interests) — not the whole store. Keeps payloads small and privacy footprint minimal.

Cloud Path (GG-Glasses-Core)

- **Session continuity:** GG-Glasses-Core keeps the Gemini session alive so follow-up queries (“and how much is it?”) can reference the prior crop without re-uploading the image.
- **Auth:** All cloud-bound auth is centralized here. Glasses never hold cloud credentials directly.

Privacy & Failure Modes

- **Privacy:** PDE stays on-device; only request-relevant slices leave the phone. Crops are ephemeral by default — not persisted unless the user opts in.
- **Offline / cloud failure:** If the cloud is unreachable, fall back to a smaller on-device VLM for a basic answer, or queue the query and notify the user when results land.

- **Latency budget:** Gesture-detect → crop → gRPC to mobile → SSGC bundle → GGGC → cloud → response, target ~1–2.5 s end-to-end. Each hop should have a stated budget so regressions are easy to localize.

Idea 2 — Mark Car Parking Spot

Goal

Help a user wearing Glasses remember exactly where they parked their car. While the user is driving, the system passively recognises the driving state through ambient understanding (vision + phone sensors). When the car is parked, it automatically captures a small set of contextual photos and extracts useful landmarks (level, section, pillar number, nearby signage). When the user later asks “Where’s my car?”, the system surfaces a clear visual + spoken summary of the saved marker.

Why It Is Useful

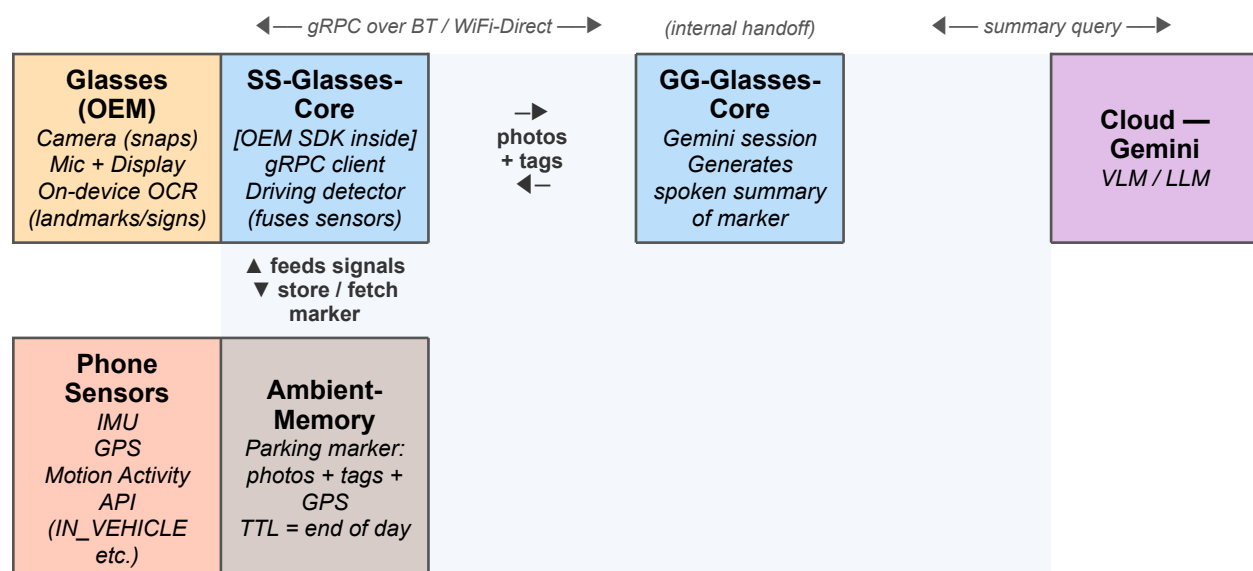
Forgetting where you parked is a small but recurring frustration — especially in multi-level garages, large outdoor lots, and unfamiliar neighbourhoods. Phone-based parking apps exist but require the user to remember to open them and tap “save”. With Glasses + always-on ambient understanding, the marker is captured automatically at the right moment, from a first-person viewpoint that matches what the user will see on return.

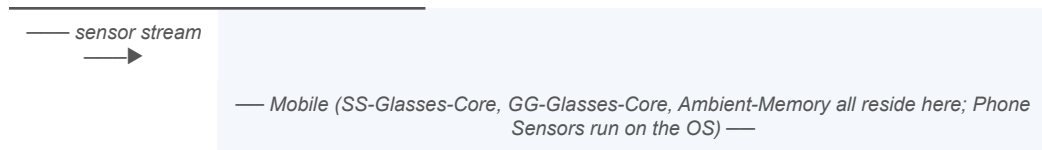
Use Cases

- **Mall / airport parking garage:** User drives to a multi-level garage, parks on Level 3 near pillar B47. The system auto-captures the level signage, pillar label, and nearby distinctive features. Hours later, “Where’s my car?” returns a spoken summary plus the photos.
- **Street parking in an unfamiliar area:** User parks on a side street in a city they don’t know well. The system captures street signs, nearby shopfronts, and coarse GPS — enough to relocate the car even without a clean GPS fix on return.

Architecture Diagram (Editable Word Shapes)

The diagram shows two related flows on the same components. Capture phase: phone sensors → driving / park detection → Glasses snaps → tags + photos written to Ambient-Memory. Retrieval phase: voice command → read Ambient-Memory → Gemini summary → audio + photos back to Glasses.





Legend

	Glasses (on-device)
	Mobile modules
	Phone Sensors (OS APIs)
	Memory module (mobile)
	Cloud services
	Watch (n/a for this idea)

Architecture Diagram (Mermaid Source)

Paste into mermaid.live, [Notion](#), [GitHub](#), or any Mermaid-compatible renderer.

```

flowchart LR
    User([User])

    subgraph GlassesNode["Glasses (OEM)"]
        direction TB
        Cam["Camera (occasional snaps)"]
        OCR["On-device OCR + landmark detect"]
        MicDisp["Mic + Display"]
    end

    subgraph PhoneSensors["Phone Sensors (OS)"]
        direction TB
        IMU["IMU"]
        GPS["GPS"]
        MAA["Motion Activity API<br/>(IN_VEHICLE / STILL)"]
    end

    subgraph Mobile["Mobile"]
        direction TB
        subgraph SSGC["SS-Glasses-Core"]
            OEM["OEM SDK"]
            DDET["Driving detector<br/>(sensor + vision fusion)"]
            PDET["Park-event detector"]
        end
        GGGC["GG-Glasses-Core<br/>(Gemini session)"]
        AMEM["Ambient-Memory<br/>parking marker:<br/>photos + tags + GPS<br/>TTL = end of day"]
    end

    Cloud["Cloud: Gemini VLM/LLM"]

    %% Capture phase
    PhoneSensors -- "sensor stream" --> DDET
    Cam -- "low-freq snaps<br/>(during drive: confirm driving)" --> OEM
    Cam -- "post-park snaps<br/>(environment capture)" --> OEM
    OEM -- "photos + extracted tags" --> SSGC
    SSGC -- "write parking marker" --> AMEM

    %% Retrieval phase
    User -- "Where's my car?<br/>(voice via Glasses mic)" --> MicDisp
    MicDisp -- "voice query (gRPC)" --> OEM
    SSGC -- "read marker" --> AMEM
    SSGC -- "marker bundle" --> GGGC
    GGGC -- "summarise marker" --> Cloud
    Cloud -- "spoken summary +<br/>photo selection" --> GGGC
    GGGC -- "summary" --> SSGC
    SSGC -- "audio + photos (gRPC)" --> MicDisp

```

Architecture Diagram (Eraser.io DSL)

Paste into *app.eraser.io* as a *Diagram-as-Code* source.

```
// Idea 2 — Mark Car Parking Spot
// Paste into https://app.eraser.io

User [icon: user, color: yellow]

Glasses (OEM) [icon: glasses, color: orange] {
  Camera [icon: camera, label: "Camera (occasional snaps)"]
  OnDevice_OCR [icon: eye, label: "OCR + landmark detect"]
  Mic_Display [icon: mic]
}

Phone_Sensors [icon: activity, color: red, label: "Phone Sensors (OS)"] {
  IMU [icon: move]
  GPS [icon: map-pin]
  Motion_Activity [icon: car, label: "Motion Activity API\nIN_VEHICLE / STILL"]
}

Mobile [icon: smartphone, color: blue] {
  SS_Glasses_Core [icon: cog, label: "SS-Glasses-Core"] {
    OEM_SDK [icon: plug]
    Driving_Detector [icon: alert-circle, label: "Driving detector"]
    Park_Detector [icon: parking-circle, label: "Park-event detector"]
  }
  GG_Glasses_Core [icon: cog, label: "GG-Glasses-Core\nGemini session"]
  Ambient_Memory [icon: database, color: brown, label: "Ambient-Memory\nparking marker\nTTL = end of day"]
}

Cloud [icon: cloud, color: purple] {
  Gemini [icon: sparkles, label: "Gemini VLM/LLM"]
}

// Capture phase
Phone_Sensors > Driving_Detector: sensor stream
Camera > OEM_SDK: low-freq snaps (gRPC over BT / WiFi-Direct)
OEM_SDK > SS_Glasses_Core: photos + extracted tags
SS_Glasses_Core > Ambient_Memory: write parking marker

// Retrieval phase
User > Mic_Display: "Where's my car?"
Mic_Display > OEM_SDK: voice query (gRPC)
SS_Glasses_Core > Ambient_Memory: read marker
SS_Glasses_Core > GG_Glasses_Core: marker bundle
GG_Glasses_Core > Gemini: summarise marker
Gemini > GG_Glasses_Core: spoken summary + photo selection
GG_Glasses_Core > SS_Glasses_Core: summary
SS_Glasses_Core > Mic_Display: audio + photos (gRPC)
```

End-to-End Flow

Capture phase

- Phone Sensors (IMU, GPS, Motion Activity API) stream signals to SS-Glasses-Core continuously at low cost.

- Driving detector in SS-Glasses-Core looks for IN_VEHICLE state + sustained speed. When triggered, it asks Glasses for occasional low-frequency snaps (e.g., one every ~30 s) to confirm the user is actually driving (steering wheel / road view) and not just a passenger or on a train.
- Park-event detector watches for the transition: motion activity drops to STILL/ON_FOOT and GPS speed reads ~0 for a sustained window (~30 s). This signals “user has parked”.
- On park-event, SS-Glasses-Core requests Glasses to run a short higher-rate capture window (~10–30 s at 1 FPS) of the surrounding environment.
- Glasses run on-device OCR + landmark detection on those frames, extracting structured tags: level/floor, section, pillar number, nearby signage, shopfronts.
- Photos + tags + GPS are returned to SS-Glasses-Core over gRPC and written as a single parking-marker entry into Ambient-Memory with TTL = end of day.

Retrieval phase

- User says “Where’s my car?” into the Glasses mic.
- Voice query → SS-Glasses-Core via gRPC; SS-Glasses-Core fetches the latest parking marker from Ambient-Memory.
- SS-Glasses-Core hands the marker bundle (photos + tags + GPS) to GG-Glasses-Core, which asks Gemini to produce a natural-language summary and select the best photo(s).
- Response flows back: Cloud → GG-Glasses-Core → SS-Glasses-Core → Glasses for audio playback and on-display photo preview. The phone app can also display the full set if the user opens it.

Technical Details to Consider

Ambient Driving Detection

- **Sensor fusion:** Motion Activity API (IN_VEHICLE) is the primary trigger; IMU + GPS speed are corroborating signals. Vision is a tie-breaker — confirms driver vs passenger via steering-wheel / dashboard cues from the Glasses camera.
- **Glasses camera duty cycle:** During the drive the camera is mostly off; it wakes up only every ~30 s for a single confirmation snap. This keeps battery / thermal cost low compared to Idea 1’s continuous 1 FPS.
- **False positives:** Train rides, buses, and being a passenger all look like IN_VEHICLE. Vision (steering wheel visible + forward-road view) is what distinguishes “driving” from “riding”. Conservative bias: skip the capture phase if driver-confirmation is uncertain.

Park-Event Detection

- **Transition signature:** Motion Activity: IN_VEHICLE → STILL or ON_FOOT, sustained for ~30 s, with GPS speed ~0. Optionally, BT disconnect from car stereo as an extra signal.
- **Capture window:** ~10–30 s of 1 FPS snaps starting at park-event. Window should be long enough to cover walking 10–20 m from the car (so we capture the user looking back at it + nearby signage).
- **Snap selection:** Keep 2–4 best photos: one of the car itself if visible, one or two of contextual landmarks (level sign, pillar marker, shopfront), and one wider view. Discard blurry/duplicate frames.

On-Glasses Extraction

- **OCR on signage:** On-device OCR extracts pillar numbers, level signs, section letters, street signs. Lightweight model — no cloud round-trip during capture.
- **Landmark detection:** Detect distinctive shapes/colours (e.g., red exit sign, painted pillar) that can be described back to the user in plain language.
- **Structured tags:** Output a small JSON: {level, section, pillar, nearby_text[], coarse_gps, capture_time}. This is what Gemini will phrase into a sentence on retrieval.

Ambient-Memory Module

- **Purpose:** New mobile-side module, distinct from PDE. PDE holds long-lived user preferences and identity; Ambient-Memory holds ephemeral situational context. SS-Glasses-Core both writes and reads it. Future ideas may also use it.
- **Schema (this idea):** parking_marker { photos[], tags{level, section, pillar, nearby_text[]}, coarse_gps, ts, ttl }.
- **TTL & cleanup:** Default TTL = end of day; auto-purged at midnight or on next park-event (one active marker at a time). User can opt to extend or pin.
- **Privacy:** Stays on-device. Nothing leaves the phone during capture. On retrieval, only the marker bundle goes to the cloud (and only when the user actively asks).

Retrieval Path

- **Voice trigger:** “Where’s my car?” recognised by Glasses mic; phone app trigger as backup.
- **Cloud summarisation:** GG-Glasses-Core asks Gemini to phrase the marker tags into a natural sentence (“Your car is on Level 3, Section B, near pillar 47, next to the red exit sign”) and pick the most informative photo.
- **Output:** Spoken summary via Glasses speakers + photo preview on Glasses display. Full set viewable on phone app.

Cost / Power Considerations

- **Why not AR pin:** Visual relocalization (VPS-style) is expensive in compute and unreliable in parking garages where GPS is weak and visual texture is repetitive. Annotated capture is far cheaper and works in places where AR pins fail.
- **Idle drain:** Phone sensor stream is cheap; the expensive part is Glasses camera + on-device vision. Duty-cycling the camera during drive and bursting only at park-event keeps the cost bounded.
- **Bandwidth:** During capture and write, everything stays on-device (Glasses → mobile over gRPC, then mobile → Ambient-Memory). Cloud is touched only on retrieval, and even then with a few compressed photos + small JSON.

Failure Modes

- **No usable signage:** If OCR finds nothing useful (e.g., empty field), fall back to: GPS + best photo of car + relative direction. Better than nothing.
- **Missed park-event:** If Motion Activity is noisy and park-event is missed, allow a manual “save parking spot” voice command as backup.
- **Multiple cars in a day:** Only one active marker at a time — each new park-event overwrites the previous (with a brief confirm window). End-of-day TTL prevents stale markers.

Idea 3 — Visual Food Logging & Dietary Analysis

Goal

Help the user log what they eat throughout the day without lifting a finger. Glasses passively watch for eating events via low-frequency ambient vision; when one is detected, the system auto-captures the meal, runs the photo through a dedicated 3rd-party nutrition API to extract cuisine type, calories, and macros, and stores the result in Ambient-Memory. Voice responses (logged-ack, end-of-day summary, optional nudges) are phrased through Gemini and spoken back via the Glasses.

Why It Is Useful

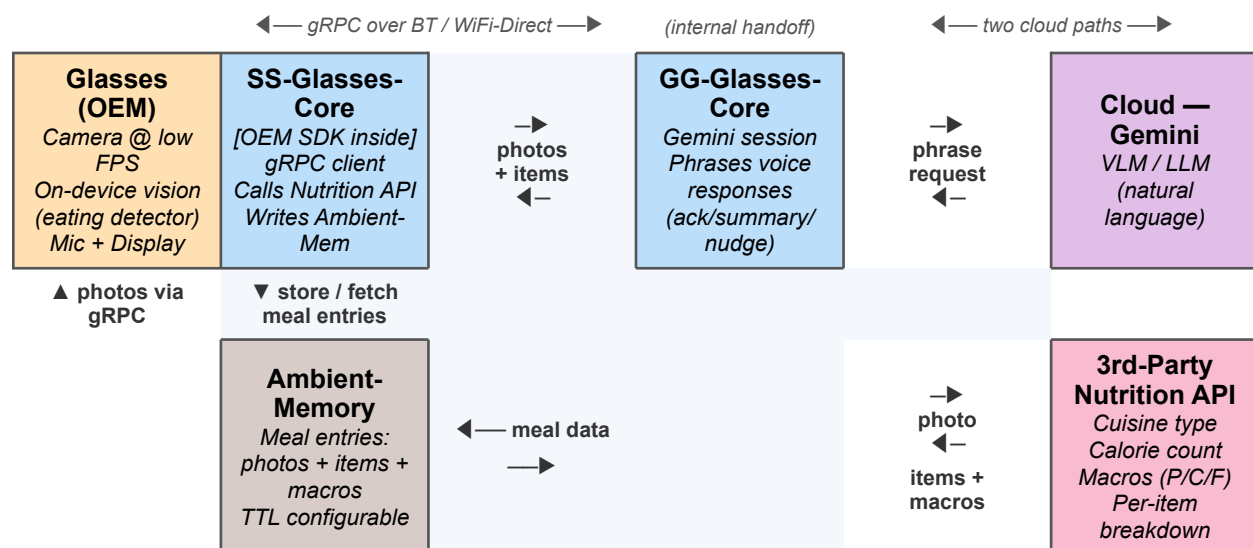
Manual food logging is the single biggest reason diet-tracking apps get abandoned — typing in every meal is tedious, and people forget. A passive, vision-driven logger captures meals at the moment they happen, from a first-person viewpoint, with no UI friction. The nutrition data is handled by a specialised 3rd-party API so the system stays independent of any single model's food knowledge, and Ambient-Memory keeps the day's history available for natural-language queries.

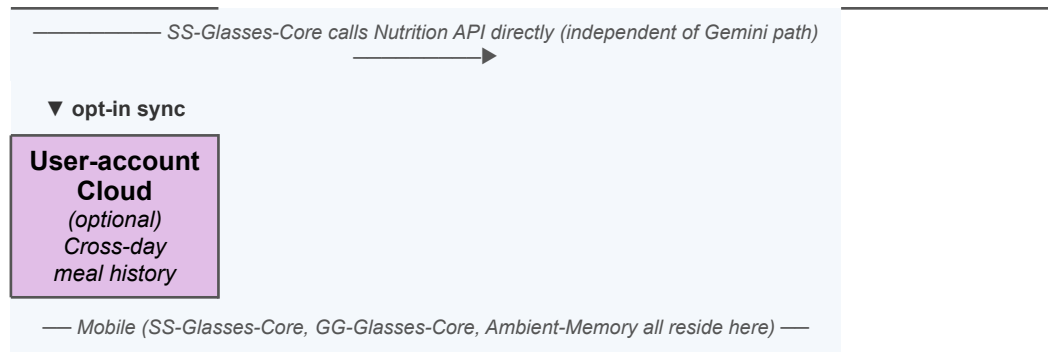
Use Cases

- **End-of-day summary:** User asks “What did I eat today?” and gets a spoken summary: “You had oatmeal with fruit for breakfast (≈320 cal), a chicken Caesar salad for lunch (≈480 cal), and salmon with rice for dinner (≈640 cal). Total ≈1,440 cal, 95 g protein.”
- **Goal-aware nudges:** Optional, opt-in. Mid-afternoon: “You’ve had about 800 cal so far — plenty of room for the dinner you have planned.” Off by default; the user chooses whether the system speaks up.

Architecture Diagram (Editable Word Shapes)

Two cloud paths run independently. The Nutrition API (3rd party, pink) takes the meal photo and returns structured nutrition data. The Gemini path (purple) handles natural-language phrasing of voice responses. Ambient-Memory holds meal entries and can optionally sync to the user's personal cloud.





Legend

	Glasses (on-device)
	Mobile modules
	Memory module (mobile)
	Cloud (Gemini / user cloud)
	3rd-Party Nutrition API
	Watch (n/a for this idea)

Architecture Diagram (Mermaid Source)

Paste into mermaid.live, [Notion](#), [GitHub](#), or any Mermaid-compatible renderer.

```

flowchart LR
    User([User])

    subgraph GlassesNode["Glasses (OEM)"]
        direction TB
        Cam["Camera @ low FPS"]
        EatDet["On-device eating detector<br/>(food/plate/utensils +<br/>hand-to-mouth motion)"]
        MicDisp["Mic + Display"]
    end

    subgraph Mobile["Mobile"]
        direction TB
        subgraph SSGC["SS-Glasses-Core"]
            OEM["OEM SDK"]
            Orchestrator["Meal-event orchestrator"]
        end
        GGGC["GG-Glasses-Core<br/>(Gemini session,<br/>phrases voice)"]
        AMEM["Ambient-Memory<br/>meal entries:<br/>photos + items + macros"]
    end

    NutritionAPI["3rd-Party Nutrition API<br/>(cuisine, calories, macros,<br/>per-item breakdown)"]
    Gemini["Cloud: Gemini VLM/LLM"]
    UserCloud["User-account Cloud<br/>(optional sync,<br/>cross-day history)"]

    %% Capture phase
    EatDet -- "eating detected (vision)" --> OEM
    Cam -- "meal photo (gRPC)" --> OEM
    SSGC -- "photo" --> NutritionAPI
    NutritionAPI -- "items + macros" --> SSGC
    SSGC -- "write meal entry" --> AMEM

    %% Voice responses
    SSGC -- "meal bundle" --> GGGC
    GGGC -- "phrase request" --> Gemini
    Gemini -- "ack/summary/nudge text" --> GGGC
    GGGC -- "voice response" --> SSGC
    SSGC -- "audio (gRPC)" --> MicDisp

    %% User query
    User -- "What did I eat today?" --> MicDisp
    MicDisp -- "voice query (gRPC)" --> OEM
    SSGC -- "fetch today's meals" --> AMEM

    %% Optional sync
    AMEM -. "opt-in sync" .-> UserCloud

```

Architecture Diagram (Eraser.io DSL)

Paste into app.eraser.io as a Diagram-as-Code source.

```
// Idea 3 — Visual Food Logging & Dietary Analysis
// Paste into https://app.eraser.io

User [icon: user, color: yellow]

Glasses (OEM) [icon: glasses, color: orange] {
  Camera [icon: camera, label: "Camera @ low FPS"]
  Eating_Detector [icon: utensils, label: "Eating detector\nfood/plate/utensils\n+
hand-to-mouth"]
  Mic_Display [icon: mic]
}

Mobile [icon: smartphone, color: blue] {
  SS_Glasses_Core [icon: cog, label: "SS-Glasses-Core"] {
    OEM_SDK [icon: plug]
    Meal_Orchestrator [icon: workflow, label: "Meal-event orchestrator"]
  }
  GG_Glasses_Core [icon: cog, label: "GG-Glasses-Core\nGemini session\nphrases
voice"]
  Ambient_Memory [icon: database, color: brown, label: "Ambient-Memory\nmeal
entries:\nphotos + items + macros"]
}

Nutrition_API [icon: bar-chart-2, color: pink, label: "3rd-Party Nutrition
API\ncouisine, calories, macros"]

Cloud [icon: cloud, color: purple] {
  Gemini [icon: sparkles, label: "Gemini VLM/LLM"]
}

User_Cloud [icon: cloud, color: purple, label: "User-account Cloud\n(opt-in sync,
\ncross-day history)"]

// Capture phase
Eating_Detector > OEM_SDK: eating detected (vision)
Camera > OEM_SDK: meal photo (gRPC over BT / WiFi-Direct)
SS_Glasses_Core > Nutrition_API: photo
Nutrition_API > SS_Glasses_Core: items + macros
SS_Glasses_Core > Ambient_Memory: write meal entry

// Voice responses
SS_Glasses_Core > GG_Glasses_Core: meal bundle
GG_Glasses_Core > Gemini: phrase request (ack/summary/nudge)
Gemini > GG_Glasses_Core: response text
GG_Glasses_Core > SS_Glasses_Core: voice response
SS_Glasses_Core > Mic_Display: audio (gRPC)

// User query
User > Mic_Display: "What did I eat today?"
Mic_Display > OEM_SDK: voice query (gRPC)
SS_Glasses_Core > Ambient_Memory: fetch today's meals

// Optional sync
Ambient_Memory > User_Cloud: opt-in sync
```

End-to-End Flow

Eating-event detection

- Glasses camera runs at low FPS (similar to Idea 1's ~1 FPS) for ambient understanding.
- An on-device eating detector looks for cues: food on a plate, utensils in hand, repeated hand-to-mouth motion, the user seated facing food.
- Detection is biased by time of day. During lunch / dinner windows the confidence threshold is lower; outside those windows it is higher to suppress false positives.
- When eating is confidently detected, the Glasses notifies SS-Glasses-Core over gRPC and triggers a single high-quality meal photo.

Capture + nutrition analysis

- Glasses captures one representative photo (option: a second photo ~5 min in, to catch added items like drinks or dessert; off by default).
- Photo travels over gRPC (BT / WiFi-Direct) to SS-Glasses-Core on mobile.
- SS-Glasses-Core sends the photo to the 3rd-party Nutrition API and receives structured data back: cuisine type, identified items, calorie count, macros (protein / carbs / fat), and whatever else the API supports.
- SS-Glasses-Core writes a meal entry into Ambient-Memory: { photo, timestamp, items[], macros, source = nutrition_api }.

Voice ack at capture

- Immediately after the meal entry is written, SS-Glasses-Core hands the entry to GG-Glasses-Core.
- GG-Glasses-Core asks Gemini to phrase a short, conversational acknowledgment ("Logged: chicken Caesar salad, about 480 calories.").
- The phrased response flows back to SS-Glasses-Core and is played on the Glasses speakers. Brief by design — the user is mid-meal.

On-demand summary

- User asks via Glasses mic: "What did I eat today?" / "How many calories so far?" / "Did I get enough protein?"
- SS-Glasses-Core fetches today's meal entries from Ambient-Memory and bundles them.
- GG-Glasses-Core sends the bundle to Gemini with the user's query for natural-language phrasing.
- Response is spoken via Glasses. Phone app shows the photo list + macros for visual review if the user wants more detail.

Optional proactive nudges

- Off by default. When enabled, GG-Glasses-Core can trigger a short spoken nudge at user-defined times (e.g., "around 4 pm, tell me where I'm at").
- Nudges read from Ambient-Memory only — no new cloud calls unless the user asks a follow-up.

Technical Details to Consider

On-Glasses Eating Detection

- **Vision-only:** No phone-sensor fusion (per design). The detector relies on food-scene recognition and hand-to-mouth motion patterns visible from the first-person camera.

- **Time-of-day prior:** Lunch / dinner windows are configurable per user. Within window, detection threshold is lower; outside, higher. Reduces false positives from incidental food-related views (e.g., walking past a bakery).
- **Debouncing:** Once a meal is logged, suppress further triggers for ~20–30 min. Avoids logging the same meal multiple times if the user looks away and back.
- **Power profile:** Low FPS keeps cost bounded. Eating-detector model runs on-device, lightweight; only fires the full capture pipeline when confident.

Nutrition API Integration

- **Independent of Gemini:** By design, the dietary analysis comes from a specialised 3rd-party service (e.g., dedicated food-recognition + nutrition database). This isolates dietary accuracy from changes in the general-purpose VLM.
- **Call path:** SS-Glasses-Core calls the Nutrition API directly — it does not go through GG-Glasses-Core. Two cloud paths, one for nutrition data, one for natural-language phrasing.
- **Schema (returned by API):** cuisine, items[{name, portion_estimate, calories, macros}], total_calories, total_macros, confidence.
- **Failure handling:** If the Nutrition API is unreachable, queue the photo locally and retry; show the user a logged-without-macros placeholder until macros arrive.
- **Portion estimation:** Single-photo portion estimation is famously noisy. Expose confidence in the entry; let the user correct via voice (“That was a small portion”) or phone app.

Ambient-Memory Schema (this idea)

- **Entry:** meal_entry { photo_id, timestamp, items[{name, portion, calories, macros}], total_calories, total_macros, confidence, user_edits? }.
- **TTL:** Configurable. Default a few days for short-term review; cross-day history is enabled by opt-in cloud sync rather than long on-device TTL.
- **User edits:** If the user corrects a logged item, the edit is persisted alongside the original so we can learn over time (without rewriting history).

Voice Response Phrasing (GG-Glasses-Core)

- **Three modes:** (1) immediate ack at capture — short and warm; (2) on-demand summary — fuller, structured; (3) optional nudge — brief, non-judgemental.
- **Tone:** Neutral and informative. No moralising about food choices. The system reports; the user decides.
- **Session reuse:** Gemini session is kept open across follow-ups (“and how much protein?” works without re-bundling everything).

Cloud Sync (Opt-in)

- **Default off:** All meal data stays on-device unless the user opts in.
- **Sync target:** The user’s own cloud account (e.g., personal Google account / Anthropic account). Not a shared service-side database.
- **Use cases for sync:** Cross-device review on phone / laptop, longer-term trends, export to other diet apps.

Privacy

- **On-device default:** Photos and meal entries live in Ambient-Memory on mobile. The only cloud traffic is the per-meal Nutrition API call (one photo) and the per-response Gemini call (small text + minimal context).
- **No always-on upload:** Even with low-FPS ambient capture, photos go to the cloud only when a meal event has fired and the user has not disabled nutrition lookup.

- **User-account sync:** Where sync is enabled, the destination is the user's own cloud account — not a third party's database — to keep the user's dietary data under their own control.

Failure Modes

- **Missed eating event:** Allow a manual voice trigger ("Log this meal") as backup. Glasses then capture and the rest of the pipeline runs identically.
- **Wrong identification:** User can correct via voice ("That was a tuna sandwich, not chicken"). Correction stored as a user_edit in the same Ambient-Memory entry.
- **Nutrition API offline:** Photo + timestamp still logged; macros backfilled when API is reachable again.

Idea 4 — Conversation Starter (Remember Who You’ve Met)

Goal

When the wearer is in conversation with another person, passively record the encounter (face vector + transcript), summarise the conversation via a text-only cloud call to Gemini, and store the result locally. When the wearer meets the same person again — even months later — surface a short reminder via an audio cue: who they are, when / where you last met, what you talked about, and any open follow-ups.

Why It Is Useful

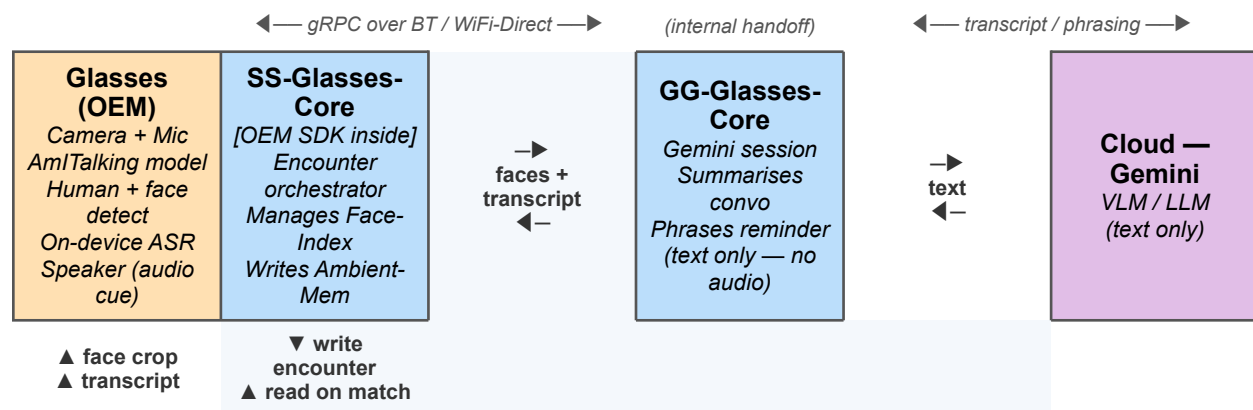
Most people are bad at remembering names and details from chance encounters. At conferences, parties, or in any role with frequent introductions, the embarrassment of “I know I’ve met you, but I can’t place you” is universal. A passive system that links a face to a small bundle of conversational context lets the wearer pick up exactly where the previous conversation left off — with no UI friction, no notebooks, no awkward asking for names twice.

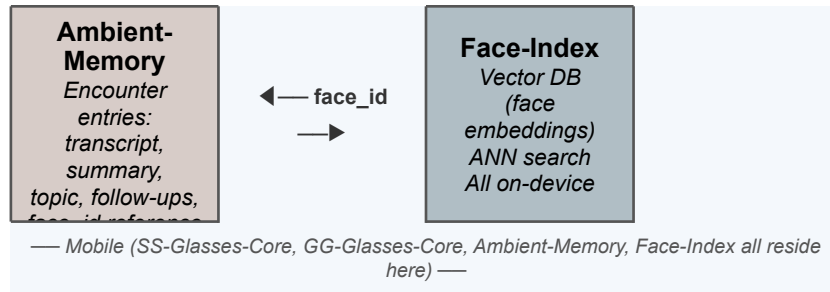
Use Cases

- **Networking events:** Wearer met someone at a conference, talked about their startup, agreed to follow up. Two months later they bump into each other again — the wearer gets an audio cue: “You met Priya at AI Summit in March. She runs a robotics startup; you said you’d intro her to Sam.”
- **Social / community settings:** Parent at a school event recognises another parent’s face but not their name or kid’s name. The system reminds them: “You met Tom at last year’s school picnic — his daughter Maya is in your son’s class.”

Architecture Diagram (Editable Word Shapes)

Two flows share components. First encounter: *AmITalking* wakes *Vision*; face + transcript are captured on-device; transcript (text only) is summarised by Gemini; the encounter is written to *Ambient-Memory* with a *Face-Index* reference. Re-encounter: new face embedding is matched against *Face-Index*; if found, the stored encounter is fetched, Gemini phrases a short blended reminder, and an audio cue plays on Glasses.





Legend

	Glasses (on-device)
	Mobile modules
	Ambient-Memory (mobile)
	Face-Index (vector DB, mobile)
	Cloud (Gemini, text only)
	Watch (n/a for this idea)

Architecture Diagram (Mermaid Source)

Paste into mermaid.live, [Notion](#), [GitHub](#), or any Mermaid-compatible renderer.

```

flowchart LR
    User([Wearer])

    subgraph GlassesNode["Glasses (OEM)"]
        direction TB
        Mic["Mic (always-on, low cost)"]
        AmI["AmITalking model<br/>(wearer / non-wearer / noise)"]
        Cam["Camera (woken on talk event)"]
        HFD["Human + face detection"]
        ASR["On-device ASR<br/>(transcribe first ~1 min,<br/>configurable)"]
        Spk["Speaker (audio cue on match)"]
    end

    subgraph Mobile["Mobile"]
        direction TB
        subgraph SSGC["SS-Glasses-Core"]
            OEM["OEM SDK"]
            EncOrch["Encounter orchestrator"]
        end
        GGGC["GG-Glasses-Core<br/>(Gemini session,<br/>text-only)"]
        AMEM["Ambient-Memory<br/>encounter entries:<br/>transcript, summary,<br/>topic, follow-ups,<br/>face_id reference"]
        FACE["Face-Index<br/>vector DB<br/>(face embeddings,<br/>ANN search, on-device)"]
    end

    Gemini["Cloud: Gemini VLM/LLM<br/>(text only - no audio,<br/>no face vectors)"]

    %% First encounter
    Mic --> AmI
    AmI -- "talk detected<br/>(wearer or non-wearer)" --> Cam
    Cam --> HFD
    HFD -- "face crop → embedding" --> OEM
    Mic --> ASR
    ASR -- "transcript chunks" --> OEM
    SSGC -- "store embedding" --> FACE
    SSGC -- "transcript bundle" --> GGGC
    GGGC -- "text only" --> Gemini
    Gemini -- "summary, name(?),<br/>topic, follow-ups" --> GGGC
    GGGC -- "encounter record" --> SSGC
    SSGC -- "write encounter (face_id ref)" --> AMEM

    %% Re-encounter
    HFD -- "new face embedding" --> OEM
    SSGC -- "ANN search" --> FACE
    FACE -- "match: face_id" --> SSGC
    SSGC -- "fetch encounter by face_id" --> AMEM
    SSGC -- "context bundle" --> GGGC
    GGGC -- "phrase reminder" --> Gemini
    Gemini -- "short blended sentence" --> GGGC
    GGGC -- "spoken reminder" --> SSGC
    SSGC -- "audio cue (gRPC)" --> Spk

```

Architecture Diagram (Eraser.io DSL)

Paste into app.eraser.io as a Diagram-as-Code source.

```
// Idea 4 — Conversation Starter (Remember Who You've Met)
// Paste into https://app.eraser.io

User [icon: user, color: yellow, label: "Wearer"]

Glasses (OEM) [icon: glasses, color: orange] {
  Mic [icon: mic, label: "Mic (always-on)"]
  AmITalking [icon: activity, label: "AmITalking model\nwearer / non-wearer /
noise"]
  Camera [icon: camera, label: "Camera (woken on talk)"]
  Human_Face_Det [icon: eye, label: "Human + face detection"]
  ASR [icon: file-text, label: "On-device ASR\n(transcribe ~1 min, configurable)"]
  Speaker [icon: volume-2, label: "Speaker (audio cue)"]
}

Mobile [icon: smartphone, color: blue] {
  SS_Glasses_Core [icon: cog, label: "SS-Glasses-Core"] {
    OEM_SDK [icon: plug]
    Encounter_Orchestrator [icon: workflow, label: "Encounter orchestrator"]
  }
  GG_Glasses_Core [icon: cog, label: "GG-Glasses-Core\nGemini session (text only)"]
  Ambient_Memory [icon: database, color: brown, label: "Ambient-Memory\nencounter
entries:\ntranscript, summary,\ntopic, follow-ups,\nface_id reference"]
  Face_Index [icon: layers, color: grey, label: "Face-Index\nvector DB\n(face
embeddings)\nANN search, on-device"]
}

Cloud [icon: cloud, color: purple] {
  Gemini [icon: sparkles, label: "Gemini\ntext only — no audio,\nno face vectors"]
}

// First encounter
Mic > AmITalking
AmITalking > Camera: talk detected (wearer or non-wearer)
Camera > Human_Face_Det
Human_Face_Det > OEM_SDK: face crop + embedding (gRPC)
Mic > ASR
ASR > OEM_SDK: transcript chunks (gRPC)
SS_Glasses_Core > Face_Index: store embedding
SS_Glasses_Core > GG_Glasses_Core: transcript bundle
GG_Glasses_Core > Gemini: text only
Gemini > GG_Glasses_Core: summary, name, topic, follow-ups
GG_Glasses_Core > SS_Glasses_Core: encounter record
SS_Glasses_Core > Ambient_Memory: write encounter (face_id ref)

// Re-encounter
Human_Face_Det > OEM_SDK: new face embedding
SS_Glasses_Core > Face_Index: ANN search
Face_Index > SS_Glasses_Core: match face_id
SS_Glasses_Core > Ambient_Memory: fetch encounter by face_id
SS_Glasses_Core > GG_Glasses_Core: context bundle
GG_Glasses_Core > Gemini: phrase reminder
Gemini > GG_Glasses_Core: short blended sentence
GG_Glasses_Core > SS_Glasses_Core: spoken reminder
SS_Glasses_Core > Speaker: audio cue (gRPC)
```

End-to-End Flow

First Encounter — Capture

- Glasses mic runs always-on with the lightweight AmITalking model classifying each audio frame as wearer-talking, non-wearer-talking, or noise.
- When either wearer-talking or non-wearer-talking fires, the Vision module is woken: camera + human-detection + face-detection.
- Detected faces are cropped and converted to embeddings on-device. On-device ASR transcribes the audio (default first ~1 minute, configurable up to entire conversation).
- Face embeddings are stored in Face-Index (vector DB, on-device). Transcript chunks flow to SS-Glasses-Core via gRPC.
- SS-Glasses-Core hands the transcript bundle to GG-Glasses-Core, which sends text only to Gemini for summarisation. Gemini returns: identified name (if mentioned), 1-line topic summary, follow-up items, relationship hint.
- SS-Glasses-Core writes an encounter entry to Ambient-Memory with a face_id reference back into Face-Index. Audio and face vectors never leave the device.

Re-encounter — Reminder

- Camera detects a face during normal ambient operation; the new embedding is matched against Face-Index via ANN search.
- On match, SS-Glasses-Core fetches the prior encounter entry from Ambient-Memory by face_id.
- The context bundle goes to GG-Glasses-Core → Gemini for natural-language phrasing of a short blended reminder.
- Reminder is played as an audio cue on the Glasses speakers — brief, low volume, designed not to be overheard.

Technical Details to Consider

AmITalking Model

- **Classes:** wearer-talking / non-wearer-talking / noise. Lightweight enough to run always-on with minimal power draw.
- **Wake conditions:** Vision wakes on either wearer-talking or non-wearer-talking. The wearer might be doing most of the talking, but the encounter is still real.
- **Hysteresis:** Brief gaps in talking don't end the encounter. Use a tail-out window (~10–20 s of silence) before closing the recording.
- **Confidence threshold:** Set high enough that ambient TV / podcasts / background chatter don't trigger. Tune per environment.

Human + Face Detection

- **Two-stage:** Human detection first (cheaper bounding box check), then face detection only inside human boxes. Avoids running expensive face models on empty frames.
- **Quality gating:** Only embed faces above a quality threshold (size, sharpness, frontal angle). Bad embeddings poison the index.
- **Multiple faces:** In a group conversation, multiple face_ids may be linked to one encounter entry. The transcript is shared; only the face references differ.

Face-Index Module

- **Why a separate module:** Vector search has different storage and query patterns than the JSON key-value Ambient-Memory. A purpose-built ANN structure (e.g., HNSW-style index) gives fast nearest-neighbour matching as the index grows.

- **Linkage:** Ambient-Memory holds the encounter content and references face_ids; Face-Index holds embeddings keyed by face_id. Clean separation of concerns.
- **On-device only:** Face vectors never leave the device. No cloud upload, no backup to user-account cloud either. This is the strongest privacy posture we can offer.
- **Match thresholds:** Require strong similarity for a match — false positives on a re-encounter are far worse than false negatives. Better to miss a reminder than to greet a stranger by the wrong name.
- **Index size & pruning:** Bound the index size. Auto-purge entries with no re-encounter after a configurable TTL (e.g., 90 days). Frequent re-encounters reset the TTL.

On-Device ASR + Cloud Summarisation

- **ASR on-device preferred:** Raw audio stays on the device; only the transcript text is sent to Gemini for summarisation. This keeps the privacy posture tight even though cloud is involved.
- **Cloud ASR as fallback:** If on-device ASR isn't available, the alternative is cloud ASR — flagged as a privacy regression. Surface this clearly in user settings and default off.
- **Configurable capture length:** Default first ~1 min; user can extend up to entire conversation, or shorten to 30 s for lighter recording.
- **Summary contents:** Gemini extracts: detected name (only if it was actually said), 1-line topic summary, follow-up items, relationship hint (colleague / friend / acquaintance / stranger — inferred from context).

Reminder Phrasing

- **Composed at re-encounter time:** Don't store a pre-baked sentence. Compose on the fly using whatever fields are available so the reminder feels current.
- **Brevity over completeness:** One short sentence. The wearer needs to act on it in seconds, not absorb a paragraph.
- **Tone:** Neutral, factual. No anxious or pushy framing ("Remember to follow up!" → "You mentioned introducing them to Sam.")

Privacy & Data Handling

- **Opt-in feature:** Off by default. The wearer must enable it explicitly.
- **Face vectors never leave device:** No cloud upload, no user-account sync. Face-Index is local-only.
- **Audio never leaves device when on-device ASR is used:** Only transcript text is sent to Gemini, and only after ASR has run locally.
- **Forget command:** Voice command "forget this person" removes the face_id from Face-Index and the matching encounter entry from Ambient-Memory.
- **Auto-purge:** TTL-based pruning of one-off encounters reduces accumulation of strangers in the index.

Drawbacks & Risks

- **Audio cue is socially awkward:** Even at low volume, having Glasses speak in front of the very person you're meeting can be uncomfortable for both parties. Mitigations: bone-conduction speaker, very brief phrasing, an opt-in haptic-only mode (would re-require Watch, currently out of scope).
- **Voice query alternative is also awkward:** Asking the Glasses "who is this?" out loud in front of the person is worse. Proactive audio cue is the least-bad option; both are imperfect.
- **Legal / regulatory:** Recording conversations is restricted in two-party-consent jurisdictions (parts of the US, EU). The feature must respect local law; some regions may require an in-product disclaimer or simply being unavailable.

- **Ethical: stranger capture:** Even with on-device-only storage, embedding strangers' faces without their knowledge is ethically contested. On-device storage is necessary but not sufficient; thoughtful defaults (short TTL, high-confidence matching only, no surfacing of unmatched embeddings) help reduce harm.
- **Accuracy in poor conditions:** Low light, sunglasses, profile views, masks, ageing all degrade face matching. Conservative thresholds reduce false positives but increase miss rate.
- **Name accuracy:** Surfacing the wrong name is the worst failure mode. Only include a name if the transcript captured it clearly; otherwise omit and let the wearer ask.

Failure Modes

- **Background voices misclassified as encounter:** AmlTalking high threshold + face-presence requirement gate this. No face in view → no encounter recorded.
- **Same person, different lighting:** Multiple embeddings per face_id over time — store a small set and average; improves robustness without inflating the index.
- **Index drift:** Periodically re-evaluate embeddings against the current face-model version; re-embed where needed.
- **Cloud unavailable at first encounter:** Summary generation queued; embedding and raw transcript stored locally and summarised when cloud is reachable. Re-encounter reminder still works — just thinner content until summary arrives.

Idea 5 — Sensor / Service State Registry + Policy Control Layer

Goal

Give the system a clean control plane for the Glasses' sensors and services. Every sensor and service exposes its state through a registry on the Glasses; the phone reads that state and writes context-driven policies that adjust what is running and at what rate. The Glasses retain local authority through a Safety Manager that can veto unsafe or disallowed requests. All on-device — no cloud involvement.

Why It Is Useful

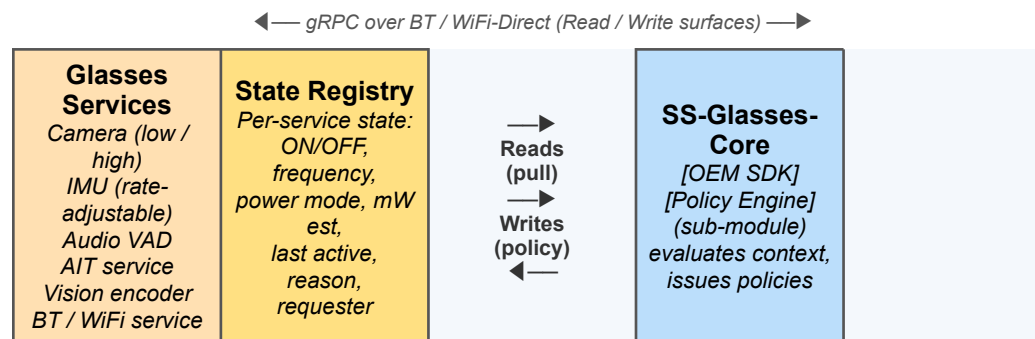
As more features are added, the Glasses end up running a growing mix of always-on, event-driven, and on-demand services — each at different rates, with different power costs, requested by different parts of the system. Without a registry, this becomes opaque: nobody knows what is running, why, or how much it costs, and it is impossible to make smart trade-offs. A state registry plus a policy engine makes the system observable and tunable. The phone, with broader context (location, activity, battery, calendar), can request the right configuration for the moment, while the Glasses retain the final say through a local safety layer.

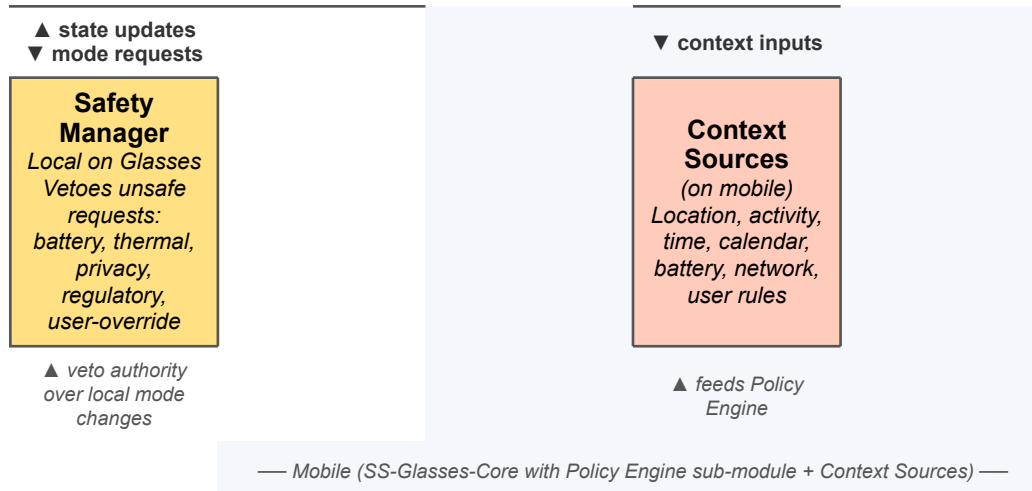
Use Cases

- **Better power management:** Dial services down when context allows and back up only when needed. Battery lasts longer without losing capability when it matters.
- **Clear debugging:** Any developer or support tool can ask the Glasses “what is running and why?” and get a structured answer.
- **Dynamic behaviour:** Same hardware, different policies for different contexts — driving, walking, in a meeting, at home, low battery.
- **Phone as smart controller, Glasses lightweight:** Heavy reasoning sits on the phone; the Glasses execute simple, well-defined commands within safety limits.
- **Easy to extend:** New sensors or services plug into the same registry contract. No bespoke control path per feature.

Architecture Diagram (Editable Word Shapes)

The State Registry sits on the Glasses next to the services it tracks. The Safety Manager (also on Glasses) has local veto authority. SS-Glasses-Core on mobile hosts the Policy Engine as a sub-module; it reads the registry by pull and writes policies. Context Sources on the mobile OS (location, activity, calendar, battery, network, user rules) feed the Policy Engine.





Legend

	Glasses services (sensor / service stack)
	State Registry + Safety Manager (Glasses)
	Mobile modules (SS-Glasses-Core + Policy Engine)
	Context Sources (OS signals on mobile)
	Cloud (n/a for this idea)
	Watch (n/a for this idea)

Architecture Diagram (Mermaid Source)

Paste into mermaid.live, Notion, GitHub, or any Mermaid-compatible renderer.

```

flowchart LR
  subgraph GlassesNode["Glasses"]
    direction TB
    subgraph Services["Glasses Services"]
      Cam["Camera (low / high)"]
      IMU["IMU (rate-adjustable)"]
      VAD["Audio VAD"]
      AIT["AIT service"]
      Enc["Vision encoder"]
      BTW["BT / WiFi service"]
    end
    end
    Registry["State Registry<br/>per-service state:<br/>ON/OFF, frequency,<br/>power mode, mW est,<br/>last active, reason,<br/>requester"]
    Safety["Safety Manager<br/>local veto:<br/>battery, thermal,<br/>privacy, regulatory,<br/>user-override"]
  end

  subgraph Mobile["Mobile"]
    direction TB
    subgraph SSGC["SS-Glasses-Core"]
      OEM["OEM SDK"]
      Policy["Policy Engine<br/>(sub-module)"]
    end
    end
    Context["Context Sources<br/>location, activity, time,<br/>calendar, battery,<br/>network, user rules"]
  end

  Services -- "state updates" --> Registry
  Registry -- "mode requests" --> Services
  Safety -- "veto authority" --> Services

  Policy -- "READ (pull): snapshot,<br/>why active, cost" --> Registry
  Policy -- "WRITE: policy<br/>(context-driven modes)" --> Registry
  Registry -- "critical events push<br/>(battery low, thermal)" --> Policy

  Context --> Policy

```

Architecture Diagram (Eraser.io DSL)

Paste into *app.eraser.io* as a *Diagram-as-Code* source.

```
// Idea 5 — Sensor / Service State Registry + Policy Control Layer
// Paste into https://app.eraser.io

Glasses [icon: glasses, color: orange] {
  Glasses_Services [icon: cpu, label: "Services"] {
    Camera [icon: camera, label: "Camera (low / high)"]
    IMU [icon: move]
    Audio_VAD [icon: mic, label: "Audio VAD"]
    AIT [icon: activity, label: "AIT service"]
    Vision_Encoder [icon: layers, label: "Vision encoder"]
    BT_WiFi [icon: bluetooth, label: "BT / WiFi service"]
  }
  State_Registry [icon: database, color: yellow, label: "State Registry\nper-service
state\nON/OFF, freq, mW est,\nreason, requester"]
  Safety_Manager [icon: shield, color: yellow, label: "Safety Manager\nlocal veto:
\nbattery, thermal, privacy,\nregulatory, user-override"]
}

Mobile [icon: smartphone, color: blue] {
  SS_Glasses_Core [icon: cog, label: "SS-Glasses-Core"] {
    OEM_SDK [icon: plug]
    Policy_Engine [icon: workflow, label: "Policy Engine\n(sub-module)"]
  }
  Context_Sources [icon: activity, color: red, label: "Context Sources\nlocation,
activity, time,\ncalendar, battery,\nnetwork, user rules"]
}

// Within Glasses
Glasses_Services > State_Registry: state updates
State_Registry > Glasses_Services: mode requests
Safety_Manager > Glasses_Services: veto authority

// Across the bridge
Policy_Engine > State_Registry: READ (pull): snapshot, why active, cost
Policy_Engine > State_Registry: WRITE: policy (context-driven modes)
State_Registry > Policy_Engine: critical events push (battery low, thermal)

// Policy inputs
Context_Sources > Policy_Engine
```

Core Concepts

1. Every sensor / service has a unique ID

Each addressable thing on the Glasses has a stable identifier. Examples:

- Camera — low-res 1 FPS
- Camera — high-res burst
- IMU 150 Hz
- Audio VAD
- AIT service
- Vision embedding encoder
- Bluetooth / Wi-Fi service

2. Each service exposes state

For every service, the registry tracks a structured record:

- ON / OFF
- Frequency / rate (e.g., 1 FPS, 150 Hz)
- Power mode (e.g., low / balanced / high)
- Current power estimate (in mW)
- Last active time
- Reason it is active
- Who requested it: Glasses, phone, user, or policy

3. Phone can READ a snapshot

The Policy Engine on the phone pulls registry data through dedicated read calls. Typical queries:

- “What is currently running?”
- “Why is the camera active?”
- “What is the battery impact right now?”
- “Can I lower IMU from 150 Hz to 30 Hz?” (asks for current cost and whether anything depends on the higher rate)

4. Phone can WRITE a policy

Policies are higher-level than direct mode changes. The Policy Engine sends a context-tagged policy, and the registry applies it across the relevant services. Examples:

- At home → reduce vision frequency
- Driving → enable IMU + audio context
- Walking outside → enable camera embedding at 1 FPS
- Meeting → enable speech mode
- Low battery → disable expensive services

Worked Example: End-of-Drive Hand-off

A concrete policy chain to make the read / write surfaces concrete.

- **Trigger:** Phone Context Sources detect a driving-session end (motion activity transitions from IN_VEHICLE to STILL, GPS speed falls to ~0 for sustained period).
- **Policy Engine reads registry:** Current state shows camera OFF (was off during drive), IMU at 150 Hz, audio VAD ON.
- **Policy Engine writes policy:** “Driving session ending → turn on ambient camera at 1 FPS for the next 60 s to enable parking-spot capture; keep IMU; AIT can drop to idle.”
- **Registry applies the policy:** Camera service is asked to move to 1 FPS low-res mode.
- **Safety Manager checks:** Battery > threshold, thermal OK, user has not toggled hard-off for camera, current location not in a restricted zone → request approved.
- **Camera service transitions, registry record updates:** { id: camera_low, state: ON, freq: 1 FPS, mode: low-power, reason: “end-of-drive ambient capture”, requester: “Policy Engine” }.
- **After 60 s the policy expires; Policy Engine writes “idle”:** Camera returns to OFF; registry record is updated.

Technical Details to Consider

Service Descriptor / Capability Discovery

- **Each service publishes a descriptor:** supported modes (e.g., camera: off / 1 FPS / 30 FPS / burst), and a cost estimate per mode (mW, bandwidth, thermal impact).

- **Why it matters:** Without descriptors the Policy Engine cannot reason about what to ask for or estimate the cost of a policy before applying it.
- **Versioning:** Descriptors carry a version so policy code on the phone can detect a Glasses firmware upgrade that added new modes.

Arbitration When Multiple Consumers Share a Sensor

- **Subscriber model:** More than one feature can ask for the same sensor (e.g., two features both want camera frames at 1 FPS). The registry holds the union of requirements.
- **Highest-demand wins:** If one consumer wants 1 FPS and another wants 5 FPS, the camera runs at 5 FPS and frames fan out to both. When the higher-demand consumer leaves, the rate drops back.
- **Fairness:** No one consumer can starve another; the registry tracks per-consumer subscriptions and tears them down explicitly when the feature ends.

Local Safety Manager

Glasses-side authority that can reject any incoming mode change. Veto reasons include:

- **Battery:** Below a threshold, expensive services cannot be turned on or up.
- **Thermal:** Sustained high-power operation pushes thermal envelope — expensive modes back off.
- **Privacy:** A camera-off hard switch set by the user is honoured regardless of any policy.
- **Regulatory:** Some venues or jurisdictions restrict camera or audio capture. Glasses can refuse requests there.
- **User override:** Direct user toggles always beat policies.
- **Bonded state:** If Glasses are not paired with the phone, only safe defaults are allowed.

Policy Precedence

- **Safety / battery / thermal policies always win:** Feature policies cannot override system-health policies.
- **User rules beat automatic context:** If the user sets “no camera at home,” no context-derived policy can flip it on.
- **Most-specific wins among feature policies:** “Driving” is more specific than “outdoors” and is applied when both fire.
- **Explicit precedence record:** Every active policy carries a precedence tag, so audits and debugging can show which one resolved a conflict.

READ Surface (Phone Pulls)

- **Snapshot:** Full registry dump — all services, their current state, their reasons, their costs.
- **Targeted queries:** “What is the camera state?” / “Why is the IMU at 150 Hz?” / “Total estimated mW now?”
- **Critical push back:** In addition to pull, the registry can push critical events the phone needs immediately (battery dropped below threshold, thermal warning, user-toggled hard-off). This keeps phone state from going stale.

WRITE Surface (Phone Sends Policies)

- **Declarative policies preferred:** “Driving → camera 1 FPS, IMU 150 Hz, audio VAD on.” The registry maps the policy onto concrete service modes.
- **Direct mode requests allowed:** Lower-level features can also ask for a specific mode (“camera 1 FPS for 30 s”). These go through the same Safety Manager.
- **Expiry / TTL:** Every policy carries an expiry. Stale policies are dropped so the system never gets stuck with a no-longer-relevant configuration.

Context Sources on Mobile

Inputs the Policy Engine consults when deciding policies:

- Location (home / work / driving / outdoors)
- Activity (motion activity API, walking, driving, still)
- Time and calendar (working hours, meetings)
- System health (battery, network availability)
- User-set rules (durable preferences, hard offs)

Failure Modes

- Glasses ↔ phone disconnected: Glasses fall back to safe defaults from the Safety Manager. Any in-flight policy expires per its TTL.
- **Policy storm:** If context flips rapidly, policies could thrash. Add minimum-dwell times before applying a policy change and hysteresis on the context signals.
- **Stale registry view on phone:** The phone's last-pulled snapshot can be stale by the time it acts. Critical-event push from Glasses helps; for non-critical reads, treat the snapshot as a hint, not ground truth, and the Glasses always make the final decision through the Safety Manager.
- **Mis-attribution:** If a service is active and the reason field is wrong (e.g., stale requester), debugging becomes hard. Require every mode change to write an updated reason atomically.

Idea 6 — Life-Logging: Ambient Memory for Humans

Goal

Build a continuously running, fully on-device contextual intelligence layer that quietly understands what the user is doing, where they are, how they feel, and what matters over time — then turns that stream into a searchable, queryable life memory.

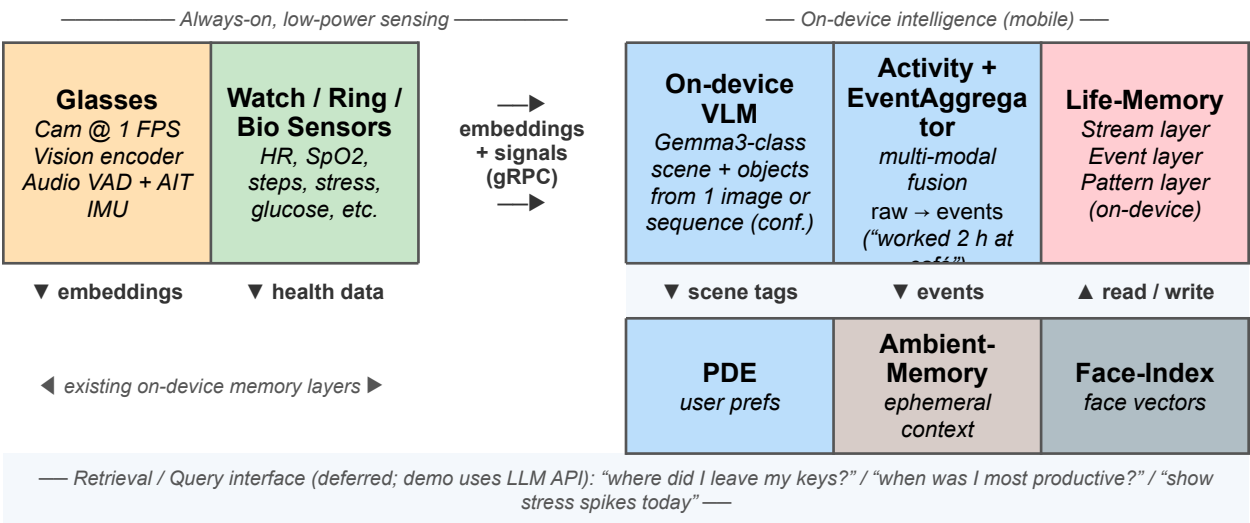
This is the capstone idea. It reuses every concept introduced so far (low-frequency vision, embedding-first transport from the General Concept; the parking-spot subsystem from Idea 2; the food-logging subsystem from Idea 3; the registry / policy control plane from Idea 5) and adds the missing piece: a persistent semantic memory of the user’s life, with three layers and a retrieval interface.

What It Captures

- **Behavioral:** working, eating, walking, driving, exercising, meeting, relaxing
- **Environmental:** home, office, gym, café, outdoors; objects nearby (laptop, coffee mug, treadmill, TV, whiteboard)
- **Physiological:** heart rate, steps, stress, sleep, SpO2, glucose, respiration, posture, fatigue
- **Temporal:** daily patterns, routines, context transitions, habit formation
- **Semantic:** life events derived from the above (“Worked intensely 2 h at café”, “High stress after meeting”, “Skipped lunch”)

Architecture Diagram (Editable Word Shapes)

Always-on sensing (Glasses + Watch / ring / bio sensors) feeds embeddings and signals to an on-device VLM (Gemma3-class) for scene understanding and an Activity / Event Aggregator for multi-modal fusion. The result lands in Life-Memory, a new on-device memory layer with three sub-layers: Stream, Event, and Pattern. Retrieval is deferred and uses an LLM API for demo only. All other components are on-device.



Legend

	Glasses (on-device)
	Watch / ring / bio sensors

	Mobile modules
	Ambient-Memory
	Face-Index
	Life-Memory (new)
	Cloud (deferred — demo only)

Architecture Diagram (Mermaid Source)

Paste into mermaid.live, [Notion](#), [GitHub](#), or any Mermaid-compatible renderer.

```

flowchart LR
    subgraph Sensing["Always-on Sensing"]
        direction TB
        Glasses["Glasses<br/>Cam @ 1 FPS, vision encoder<br/>Audio VAD + AIT, IMU"]
        Wearables["Watch / Ring / Bio sensors<br/>HR, SpO2, steps, stress,<br/>glucose, etc."]
    end

    subgraph Mobile["Mobile (on-device intelligence)"]
        direction TB
        VLM["On-device VLM<br/>(Gemma3-class)<br/>scene + objects from<br/>1 image or sequence"]
        Agg["Activity + Event Aggregator<br/>multi-modal fusion<br/>raw → semantic events"]

        subgraph LifeMem["Life-Memory (new on-device layer)"]
            direction TB
            Stream["Stream layer<br/>(raw rows / embeddings)"]
            Event["Event layer<br/>(semantic life events)"]
            Pattern["Pattern layer<br/>(routines, habits)"]
        end

        PDE["PDE<br/>user prefs"]
        AMEM["Ambient-Memory<br/>ephemeral context"]
        FACE["Face-Index<br/>face vectors"]
    end

    Glasses -- "embeddings (gRPC over BT/WiFi-Direct)" --> VLM
    Wearables -- "health signals" --> Agg
    VLM -- "scene tags" --> Agg
    Agg --> Stream
    Stream --> Event
    Event --> Pattern

    Retrieval["Retrieval / Query interface<br/>(deferred — demo uses LLM API)<br/>\"where did I leave my keys?\""]
    LifeMem -- "queryable timeline" --> Retrieval
  
```

Architecture Diagram (Eraser.io DSL)

Paste into app.eraser.io as a Diagram-as-Code source.

```
// Idea 6 — Life-Logging: Ambient Memory for Humans
// Paste into https://app.eraser.io

Sensing [icon: activity, color: orange, label: "Always-on Sensing"] {
  Glasses [icon: glasses, label: "Glasses\nCam @ 1 FPS, vision encoder\nAudio VAD + AIT, IMU"]
  Wearables [icon: watch, color: green, label: "Watch / Ring / Bio\nHR, SpO2, steps, stress, glucose"]
}

Mobile [icon: smartphone, color: blue, label: "Mobile (on-device intelligence)"] {
  On_Device_VLM [icon: sparkles, label: "On-device VLM\n(Gemma3-class)\nscene + objects"]
  Aggregator [icon: workflow, label: "Activity + Event Aggregator\nmulti-modal fusion\nraw → events"]

  Life_Memory [icon: database, color: red, label: "Life-Memory (new)"] {
    Stream_Layer [icon: layers, label: "Stream layer"]
    Event_Layer [icon: list, label: "Event layer"]
    Pattern_Layer [icon: trending-up, label: "Pattern layer"]
  }

  PDE [icon: database, label: "PDE (existing)"]
  Ambient_Memory [icon: database, color: brown, label: "Ambient-Memory (existing)"]
  Face_Index [icon: layers, color: grey, label: "Face-Index (existing)"]
}

Glasses > On_Device_VLM: embeddings (gRPC over BT/WiFi-Direct)
Wearables > Aggregator: health signals
On_Device_VLM > Aggregator: scene tags
Aggregator > Stream_Layer
Stream_Layer > Event_Layer
Event_Layer > Pattern_Layer

Retrieval [icon: search, color: purple, label: "Retrieval / Query (deferred)\ndemo uses LLM API\n\"where did I leave my keys?\""]
Life_Memory > Retrieval: queryable timeline
```

Three-Layer Memory

- **Stream layer:** raw rows at fixed cadence (e.g., every 5 min) holding embeddings + signals. Cheap to write, expensive to query directly. Bounded retention.
- **Event layer:** semantic life events aggregated from stream rows (“worked 2 h at café”, “gym session, high HR”, “parked at Level 3 B47”, “lunch ≈ 480 cal”). This is the layer humans want to query.
- **Pattern layer:** longitudinal patterns and routines extracted from events (“walks less on WFH days”, “sleep worse after late caffeine”, “stress spikes during afternoon meetings”).

Life-Memory — Where It Sits

- **New on-device memory layer:** sits next to PDE, Ambient-Memory, and Face-Index. Distinct purpose: persistent semantic timeline.
- **Boundaries:** PDE = stable user preferences; Ambient-Memory = ephemeral short-lived context; Face-Index = face vectors; Life-Memory = long-lived semantic life timeline.
- **Existing subsystems plug in:** the parking-marker entries from Idea 2 and meal entries from Idea 3 are specialised event-layer producers — they show up as native events in this timeline.

Synthetic Dataset Snippet

Stream-layer rows. Highlighted rows demonstrate how Idea 2 (parking marker) and Idea 3 (food log) entries integrate as native events. Columns truncated for fit — a real row also carries vision / audio / activity embeddings.

Time	Activity	Environment	Prominent Objects	HR bpm	Steps	kcal	SpO ₂ %	Stress
6:00	Wake / hygiene	Home — bedroom	bed, phone, mirror	59	3	5	98	24
6:30	Stretch / meditation	Home — living room	yoga mat, cushion	71	72	42	99	14
7:00	Breakfast	Home — kitchen	coffee, eggs, toast	80	352	98	99	24
7:30	Commute / walk	Street	car, crosswalk, backpack	94	721	166	98	38
8:00	Coffee + planning	Café	laptop, mug, notebook	74	1722	254	99	21
12:30	Lunch (logged)	Café	salad bowl, fork, water	78	4210	612	98	22
12:35	→ Food log event	(Idea 3 subsystem)	Caesar salad ≈ 480 cal, 32 g protein	—	—	—	—	—
14:00	Focused work	Office	monitor, keyboard, mug	72	5104	812	99	31
15:30	Meeting	Office — room 4	whiteboard, laptop, chair	84	5380	920	98	58
17:45	Drive home	Car	steering wheel, dashboard	82	6112	1080	98	34
18:05	Park (logged)	Garage — Level 3	pillar B47, exit sign, car	76	6125	1086	98	26
18:06	→ Parking marker	(Idea 2 subsystem)	L3 / Sec B / pillar 47 + photos	—	—	—	—	—
19:00	Gym session	Gym	treadmill, towel, water	138	8950	1410	97	44
21:00	Wind-down	Home — living room	couch, TV, lamp	68	9210	1520	98	18

Sample Retrieval Queries (deferred — demo via LLM API)

The Retrieval / Query interface is not part of this design pass. For demo, point an LLM API at the Life-Memory contents and ask:

- **Health:** “When did my stress spike today, and what was I doing?”
- **Health + food:** “How does my glucose respond to lunch on café days vs office days?”
- **Habits:** “Which days do I walk the least, and why?”
- **Productivity:** “When was I most focused this week? What environments correlate with focus?”
- **Spatial recall:** “Where did I leave my keys?” / “Which café was I at last Tuesday?”

Technical Details to Consider

Sensing (always-on, low-power)

- **Glasses:** camera at 1 FPS; on-device vision encoder produces compact embeddings (not raw frames) sent over gRPC; audio VAD + AIT; IMU.
- **Watch / ring / bio sensors:** health-only — HR, SpO2, steps, stress, glucose; combined with food-log events for high-value insights.
- **Embedding-first transport:** per the General Concept — embeddings/signals over the air, not raw streams.

On-device VLM for scene understanding

- **Model:** Gemma3-class small VLM running on mobile.
- **Single-image inference:** one frame → scene label + object list.
- **Sequence inference:** short sequence of frames → same scene label with higher confidence + temporal cues (“walking through café” vs “seated at café”).

Activity + Event Aggregator

- **Multi-modal fusion:** combines visual embeddings, audio embeddings, IMU activity, health signals, time, location.
- **Event-triggered:** doesn’t process everything — fires on meaningful change (new environment, HR spike + speech, sustained activity).
- **Output:** structured life events written to the Event layer of Life-Memory.

Storage & Retention

- **All on-device:** no cloud upload at any layer.
- **Stream layer:** bounded TTL (e.g., 7–30 days, user-configurable).
- **Event layer:** long-lived; the semantic record.
- **Pattern layer:** compact, durable, derived from events.

Privacy

- **Default off:** feature opt-in.
- **Granular toggles:** disable health / audio / environment independently.
- **Forget commands:** “forget the last hour”, “forget today”, “forget all visits to X”.
- **No cloud upload of raw:** ever.

Why This Is Hard

- **Deciding what matters:** the system must filter meaningful moments out of a continuous stream — not just record everything.
- **Avoiding noise:** most of the day is uneventful; the value is in the few minutes that aren’t.

- **Privacy vs usefulness:** the more detail captured, the more useful the recall — and the higher the privacy stakes.
- **Meaningful retrieval:** structured storage is easy; turning natural-language queries into useful answers over a multi-modal timeline is the real work.
- **Data overload:** the winners in this space won't be the ones who record the most — they'll be the ones who extract the most meaningful context.

General Concept — Glasses as Sensor, Phone as Brain

TL;DR

Glasses should not stream raw camera or audio continuously. They should run tiny always-on models, create visual and audio embeddings, and send those small features to the phone. The phone runs heavier AI models and decides what matters.

System Idea

1. Glasses side

- Camera captures low-res frame, example: 256×256.
- Tiny vision encoder converts image → compact embedding.
- Audio model / VAD continuously checks:
 - user speaking
 - ambient sound
 - keyword / hotword
- IMU helps detect motion / context change.
- Send only embeddings / events to phone.

2. Phone side

- Runs bigger models.
- Understands scene / context.
- Combines:
 - visual embedding
 - audio embedding
 - IMU state
 - time / location
- Creates memory / action summary.

Visual Payload Comparison

For one 256×256 frame:

Type	Approx Size
Raw grayscale	~65 KB
Raw RGB	~196 KB
JPEG compressed	~10–40 KB
Visual embedding	~0.5–2 KB

So embedding can be 10× to 100× smaller than sending images.

Power Advantage

At 1 FPS:

- Sending JPEG every second may cost several mW average depending on radio / camera pipeline.

- Sending only embedding may be much lower, often closer to sub-mW to a few mW, depending on encoder cost.
- But important: the camera sensor + ISP + encoder model power must be counted, not just radio transmission.

Best architecture

Use a tiny gatekeeper model first. Only run vision embedding when scene changes, or once per second in low-power mode.

Why This Is Powerful

- **Lower bandwidth:** You avoid streaming images.
- **Better privacy:** Phone receives features, not raw photos.
- **Lower heat:** Less radio usage and less continuous heavy vision processing.
- **Longer battery life:** Glasses stay lightweight.
- **Scalable AI:** Phone can run multiple tiny / mid models:
 - scene classifier
 - object / context detector
 - activity detector
 - speech / ambient classifier
 - memory summariser

Audio Embedding Idea

Use an always-on audio model for:

- VAD / user-speaking detection
- Hotword detection
- Ambient audio understanding
- Noise level
- Environment type:
 - cafe
 - car
 - office
 - street
 - meeting room

When the user is not speaking, send ambient audio embeddings maybe once per second. When the user is speaking, increase frequency or send richer audio / stream to the phone.

Final Architecture Sentence

Glasses = sensing + tiny gatekeeper + embeddings. Phone = intelligence + heavier models + memory / action generation. That is the right split.